Extension Methods

Why Extension Method

Suppose we have a class with 10 methods.

Now this class has been through testing and got the sign off.

But suddenly we want to add 5 more methods to it. If we add them to the class then it will have 15 methods and the whole class has to again go through the entire testing process which is unnecessary.

Not only this, consider a scenario where a class does not belong to us and we want to new add methods to it. How to do?

Please see the next example that elaborates the above scenario.

Why Extension Method

```
static void Main(string[] args)
{
    string strName = "avishek";

    //Now we want a method ChangeFirstLetterCase avialable to string class
    //that changes the case of first letter
    string result = strName.ChangeFirstLetterCase();

    Console.ReadLine();
}
```

CS1061 'string' does not contain a definition for 'ChangeFirstLetterCase' and no extension method 'ChangeFirstLetterCase' accepting a first argument of type 'string' could be found (are you missing a using directive or an assembly reference?)

Here we want to add a method to string class that will change the case of first letter. But string class does not belong to us. So we cannot modify its signature to add our method.

Why Extension Method

Solutions?

1. Inherit from the class to which we want to add our new methods. Problem: The class might be sealed. Also the type to which we want to add our methods might be a struct. We cannot apply inheritance to a struct.

2. Build a wrapper class.

```
public class StringHelper
{
    public static string ChangeFirstLetterCase(string str)
    {
        if (str.Length > 0)
        {
            char[] charArray = str.ToCharArray();
            charArray[0] = char.IsUpper(charArray[0]) ?
                           char.ToLower(charArray[0]) :
                           char.ToUpper(charArray[0]);
        }
        return str;
    }
}
```

```
class MainClass
{
    static void Main(string[] args)
    {
        string strName = "avishek";

        //Now we want a method ChangeFirstLetterCase avialable to string class
        //that changes the case of first letter
        string result = StringHelper.ChangeFirstLetterCase(strName);

        Console.ReadLine();
    }
}
```

Extension Methods

- This is a new feature added to Csharp 3.0
- It’s a mechanism of adding new methods into an existing class or structure also without modifying the source code of the original type and in this process we don’t require any permission from the original type and the original type doesn’t require any recompilation.

Extension Methods

How to convert a wrapper class into an extension method?

1. Make StringHelper static class
2. The type the method extends should be passed as a first parameter with **this** keyword preceding it. Here the extension method is extending a string type(strName.) so the first parameter is string.

```
public static class StringHelper
{
    public static string ChangeFirstLetterCase(this string str)
    {
        if (str.Length > 0)
        {
            char[] charArray = str.ToCharArray();
            charArray[0] = char.IsUpper(charArray[0]) ?
                           char.ToLower(charArray[0]) :
                           char.ToUpper(charArray[0]);
        }
        return str;
    }
}

class MainClass
{
    static void Main(string[] args)
    {
        string strName = "avishek";

        //Now we want a method ChangeFirstLetterCase avialable to string class
        //that changes the case of first letter
        string result = strName.ChangeFirstLetterCase();

        Console.ReadLine();
    }
}
```

Practical use of Extension Method: LINQ

In spite of **Where()** method not belonging to **List<T>** class, we are still able to use it as though it belong to **List<T>** class. This is possible because **Where()** method is implemented as extension method in **IEnumerable<T>** interface and **List<T>** implements **IEnumerable<T>** interface.

```
using System;
using System.Text;
using System.Collections.Generic;
using System.Collections;
using System.Linq;

class MainClass
{
    static void Main(string[] args)
    {
        List<int> Numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
        IEnumerable<int> EvenNumbers = Numbers.Where(n => n % 2 == 0);

        foreach (int num in EvenNumbers)
            Console.WriteLine(num);

        Console.ReadLine();
    }
}
```

```
using System;
using System.Text;
using System.Collections.Generic;
using System.Collections;
using System.Linq;

class MainClass
{
    static void Main(string[] args)
    {
        List<int> Numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
        IEnumerable<int> EvenNumbers = Enumerable.Where(Numbers, n => n % 2 == 0);

        foreach (int num in EvenNumbers)
            Console.WriteLine(num);

        Console.ReadLine();
    }
}
```

As you can see we can use the wrapper class type syntax because Where is an extension method of Enumerable.